In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
from tensorflow.keras import models, layers
from sklearn.metrics import confusion_matrix, accuracy_score, precision_s
import seaborn as sns

import os
```

```
2026-03-13 15:18:52.368945: I tensorflow/core/platform/cpu_feature_guard.c
c:210] This TensorFlow binary is optimized to use available CPU instructio
ns in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebui
ld TensorFlow with the appropriate compiler flags.
```

# Question 1

## 1. Preprocessing

### Load the data folder

In [126…
```
inside_paths  = [os.path.join("fence_renamed/inside",  f) for f in os.lis
outside_paths = [os.path.join("fence_renamed/outside", f) for f in os.lis

# Shuffle the data
np.random.shuffle(inside_paths)
np.random.shuffle(outside_paths)
```

### Balance the binary dataset

Done to reduce classification bias

In [127…
```
n_samples = min(len(inside_paths), len(outside_paths))
inside_paths = inside_paths[:n_samples]
outside_paths = outside_paths[:n_samples]
print(f"Balanced: {n_samples} per class, {n_samples * 2} total")
```

```
Balanced: 145 per class, 290 total
```

### Load the actual images

Done using CV2. The images are then converted to grayscale and then thresholed to binary (0 and 1). The image is also resized to reduce the parameters for the model

In [128…
```
IMG_SIZE = (128, 128)

images = []

for p in inside_paths + outside_paths:
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, IMG_SIZE)
```

```
    _, img = cv2.threshold(img, 200, 255, cv2.THRESH_BINARY_INV)
    images.append(img)

images = np.array(images, dtype="float32") / 255.0
images = images[..., np.newaxis]

labels = np.array([0] * len(inside_paths) + [1] * len(outside_paths), dty
```

## Split the data into training and testing sets

In [129…

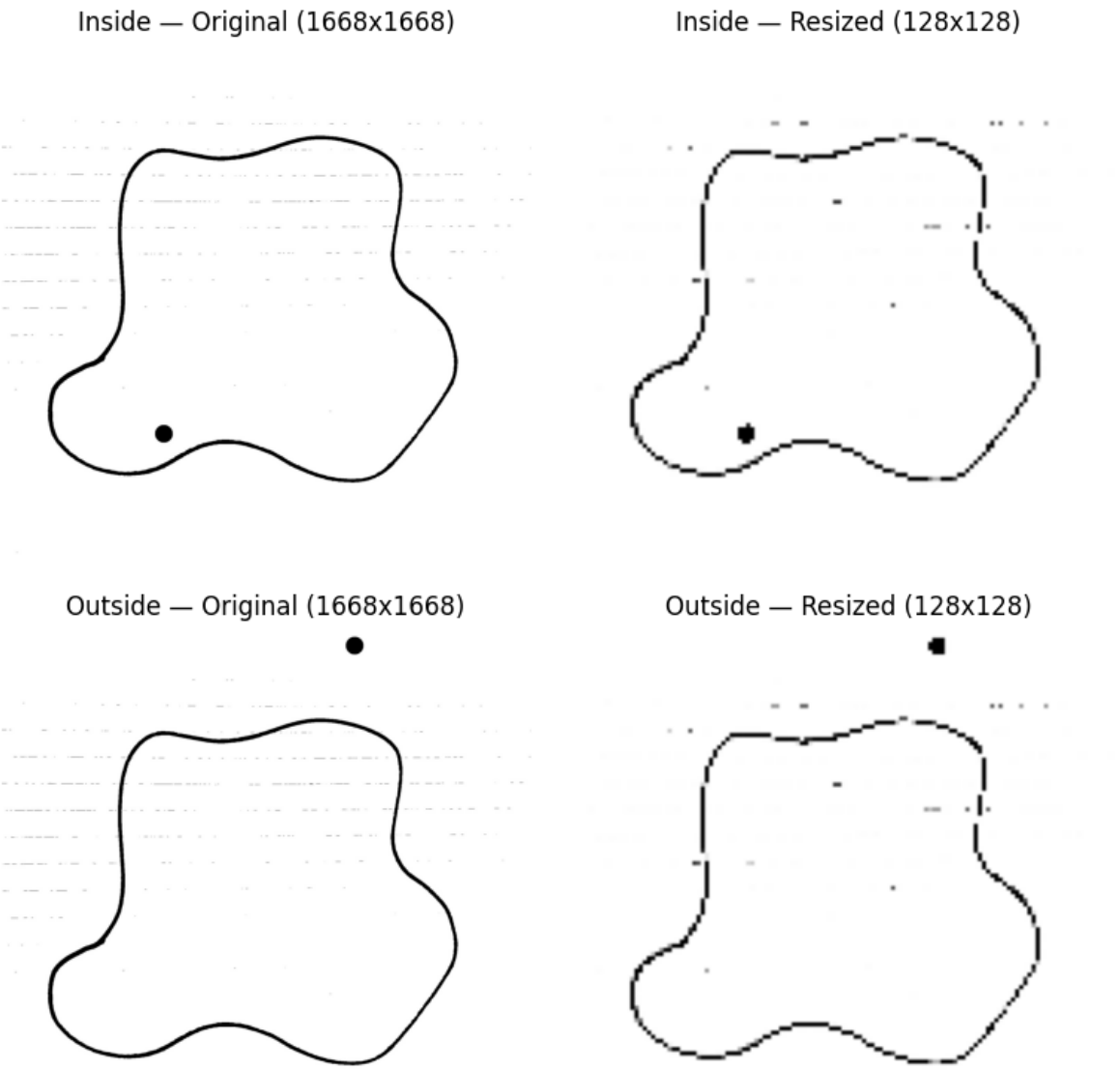
```
# shuffle
idx = np.random.permutation(len(images))
images, labels = images[idx], labels[idx]

# split
split = int(len(images) * 0.8)
train_data, test_data  = images[:split], images[split:]
train_labels, test_labels = labels[:split], labels[split:]

print(f"Total: {len(images)}\nTrain: {len(train_data)}, Test: {len(test_d
print("--------------------")
print(f"Train = Inside: {int((train_labels==0).sum())} ; Outside: {int((t
print(f"Test = Inside: {int((test_labels==0).sum())} ; Outside: {int((tes
```

```
Total: 290
Train: 232, Test: 58
--------------------
Train = Inside: 117 ; Outside: 115
Test = Inside: 28 ; Outside: 30
```

## Visualize the Images

In [130…

```
fig, axes = plt.subplots(2, 2, figsize=(8, 8))

for row, (path, label) in enumerate([(inside_paths[0], "Inside"), (outsid
    original = cv2.cvtColor(cv2.imread(path), cv2.COLOR_BGR2RGB)
    resized  = cv2.cvtColor(cv2.resize(cv2.imread(path), IMG_SIZE), cv2.C

    axes[row][0].imshow(original)
    axes[row][0].set_title(f"{label} — Original ({original.shape[1]}x{ori
    axes[row][0].axis("off")

    axes[row][1].imshow(resized)
    axes[row][1].set_title(f"{label} — Resized ({IMG_SIZE[0]}x{IMG_SIZE[1
    axes[row][1].axis("off")

plt.tight_layout()
plt.show()
```




## 2. Constructing the FCN

In [ ]:
```
model = models.Sequential([
    layers.Input(shape=(128,128,1)),
    layers.Flatten(),
    layers.Dense(32, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

history = model.fit(train_data, train_labels, epochs=100, validation_data

test_loss, test_accuracy = model.evaluate(test_data, test_labels, verbose
print(f'Test accuracy: {test_accuracy}')

train_loss = history.history['loss']
train_accuracy = history.history['accuracy']
val_loss = history.history['val_loss']
```

```
Epoch 1/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 2s 154ms/step - accuracy: 0.4784 - loss: 0.7006 -
val_accuracy: 0.4828 - val_loss: 0.7065
Epoch 2/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.5388 - loss: 0.6827 -
val_accuracy: 0.5172 - val_loss: 0.6876
Epoch 3/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.6164 - loss: 0.6693 -
val_accuracy: 0.7069 - val_loss: 0.6836
Epoch 4/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.7241 - loss: 0.6490 -
val_accuracy: 0.5345 - val_loss: 0.6775
Epoch 5/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8448 - loss: 0.6246 -
val_accuracy: 0.7759 - val_loss: 0.6699
Epoch 6/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.5856 -
val_accuracy: 0.7586 - val_loss: 0.6590
Epoch 7/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.5421 -
val_accuracy: 0.5517 - val_loss: 0.6580
Epoch 8/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.4937 -
val_accuracy: 0.7586 - val_loss: 0.6321
Epoch 9/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.4315 -
val_accuracy: 0.8103 - val_loss: 0.6140
Epoch 10/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.3765 -
val_accuracy: 0.6552 - val_loss: 0.6101
Epoch 11/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.3071 -
val_accuracy: 0.8276 - val_loss: 0.5771
Epoch 12/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.2452 -
val_accuracy: 0.7069 - val_loss: 0.5619
Epoch 13/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.1911 -
val_accuracy: 0.7241 - val_loss: 0.5416
Epoch 14/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.1459 -
val_accuracy: 0.7414 - val_loss: 0.5228
Epoch 15/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.1084 -
val_accuracy: 0.7759 - val_loss: 0.4989
Epoch 16/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.0843 -
val_accuracy: 0.7069 - val_loss: 0.4899
Epoch 17/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0651 -
val_accuracy: 0.8103 - val_loss: 0.4759
Epoch 18/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0511 -
val_accuracy: 0.8103 - val_loss: 0.4661
Epoch 19/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0425 -
val_accuracy: 0.7931 - val_loss: 0.4572
Epoch 20/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0347 -
val_accuracy: 0.8276 - val_loss: 0.4529
```

```
Epoch 21/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0294 -
val_accuracy: 0.7414 - val_loss: 0.4556
Epoch 22/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0266 -
val_accuracy: 0.7759 - val_loss: 0.4450
Epoch 23/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0231 -
val_accuracy: 0.7414 - val_loss: 0.4505
Epoch 24/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0189 -
val_accuracy: 0.8276 - val_loss: 0.4312
Epoch 25/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0160 -
val_accuracy: 0.7069 - val_loss: 0.4338
Epoch 26/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0145 -
val_accuracy: 0.8276 - val_loss: 0.4247
Epoch 27/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.0129 -
val_accuracy: 0.8276 - val_loss: 0.4254
Epoch 28/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 1.0000 - loss: 0.0117 -
val_accuracy: 0.8276 - val_loss: 0.4198
Epoch 29/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0105 -
val_accuracy: 0.8276 - val_loss: 0.4179
Epoch 30/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0096 -
val_accuracy: 0.8276 - val_loss: 0.4159
Epoch 31/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0088 -
val_accuracy: 0.8276 - val_loss: 0.4141
Epoch 32/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 0.0081 -
val_accuracy: 0.8448 - val_loss: 0.4118
Epoch 33/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.0075 -
val_accuracy: 0.8276 - val_loss: 0.4106
Epoch 34/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0069 -
val_accuracy: 0.8276 - val_loss: 0.4088
Epoch 35/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0064 -
val_accuracy: 0.8276 - val_loss: 0.4076
Epoch 36/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0061 -
val_accuracy: 0.8276 - val_loss: 0.4055
Epoch 37/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0056 -
val_accuracy: 0.8276 - val_loss: 0.4048
Epoch 38/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0052 -
val_accuracy: 0.8448 - val_loss: 0.4031
Epoch 39/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0049 -
val_accuracy: 0.8103 - val_loss: 0.4045
Epoch 40/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0047 -
val_accuracy: 0.8276 - val_loss: 0.4011
```

```
Epoch 41/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0044 -
val_accuracy: 0.8448 - val_loss: 0.3996
Epoch 42/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0041 -
val_accuracy: 0.8103 - val_loss: 0.4012
Epoch 43/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0039 -
val_accuracy: 0.8103 - val_loss: 0.3989
Epoch 44/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0037 -
val_accuracy: 0.8276 - val_loss: 0.3966
Epoch 45/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0035 -
val_accuracy: 0.8103 - val_loss: 0.3987
Epoch 46/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0034 -
val_accuracy: 0.8276 - val_loss: 0.3947
Epoch 47/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0032 -
val_accuracy: 0.7931 - val_loss: 0.3964
Epoch 48/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0030 -
val_accuracy: 0.8276 - val_loss: 0.3932
Epoch 49/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0029 -
val_accuracy: 0.8448 - val_loss: 0.3925
Epoch 50/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0028 -
val_accuracy: 0.8276 - val_loss: 0.3948
Epoch 51/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0026 -
val_accuracy: 0.8103 - val_loss: 0.3919
Epoch 52/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0025 -
val_accuracy: 0.8276 - val_loss: 0.3904
Epoch 53/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0024 -
val_accuracy: 0.8103 - val_loss: 0.3922
Epoch 54/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0023 -
val_accuracy: 0.8276 - val_loss: 0.3889
Epoch 55/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0022 -
val_accuracy: 0.8103 - val_loss: 0.3893
Epoch 56/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0021 -
val_accuracy: 0.8448 - val_loss: 0.3880
Epoch 57/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0020 -
val_accuracy: 0.8276 - val_loss: 0.3873
Epoch 58/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0020 -
val_accuracy: 0.8448 - val_loss: 0.3868
Epoch 59/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0019 -
val_accuracy: 0.8276 - val_loss: 0.3868
Epoch 60/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0018 -
val_accuracy: 0.8276 - val_loss: 0.3862
```

```
Epoch 61/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0018 -
val_accuracy: 0.8276 - val_loss: 0.3849
Epoch 62/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0017 -
val_accuracy: 0.8276 - val_loss: 0.3850
Epoch 63/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0016 -
val_accuracy: 0.8276 - val_loss: 0.3844
Epoch 64/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0016 -
val_accuracy: 0.8276 - val_loss: 0.3839
Epoch 65/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0015 -
val_accuracy: 0.8276 - val_loss: 0.3843
Epoch 66/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0015 -
val_accuracy: 0.8276 - val_loss: 0.3826
Epoch 67/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.0014 -
val_accuracy: 0.8448 - val_loss: 0.3823
Epoch 68/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0014 -
val_accuracy: 0.7931 - val_loss: 0.3839
Epoch 69/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0013 -
val_accuracy: 0.8276 - val_loss: 0.3813
Epoch 70/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0013 -
val_accuracy: 0.8276 - val_loss: 0.3810
Epoch 71/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0013 -
val_accuracy: 0.8448 - val_loss: 0.3805
Epoch 72/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0012 -
val_accuracy: 0.8276 - val_loss: 0.3801
Epoch 73/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0012 -
val_accuracy: 0.8276 - val_loss: 0.3807
Epoch 74/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0012 -
val_accuracy: 0.8103 - val_loss: 0.3803
Epoch 75/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0011 -
val_accuracy: 0.8448 - val_loss: 0.3790
Epoch 76/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0011 -
val_accuracy: 0.8276 - val_loss: 0.3786
Epoch 77/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0011 -
val_accuracy: 0.8276 - val_loss: 0.3787
Epoch 78/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0010 -
val_accuracy: 0.8448 - val_loss: 0.3781
Epoch 79/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0010 -
val_accuracy: 0.8276 - val_loss: 0.3776
Epoch 80/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 9.7202e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3780
```

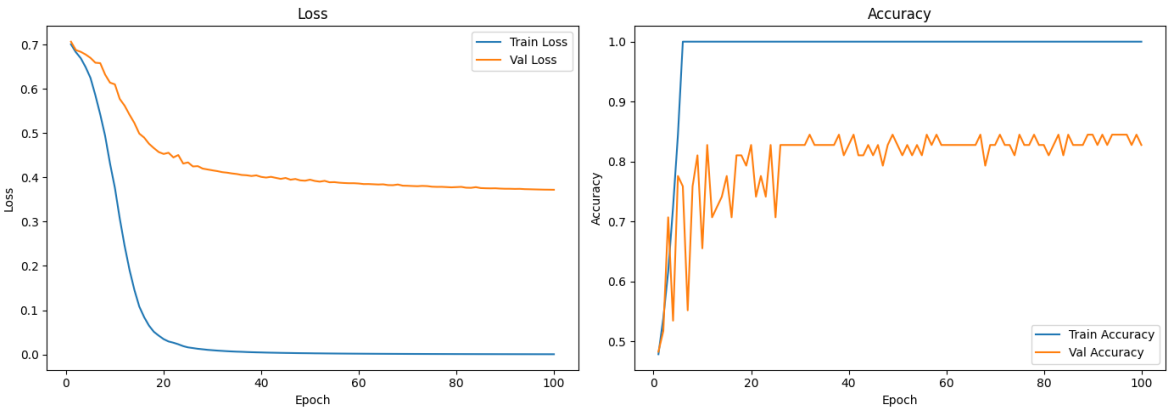
```
Epoch 81/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 9.5551e-0
4 - val_accuracy: 0.8103 - val_loss: 0.3786
Epoch 82/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 9.2564e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3767
Epoch 83/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 9.0070e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3764
Epoch 84/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 8.7855e-0
4 - val_accuracy: 0.8103 - val_loss: 0.3780
Epoch 85/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 8.5632e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3758
Epoch 86/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 8.3398e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3754
Epoch 87/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 8.1505e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3752
Epoch 88/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 7.9308e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3753
Epoch 89/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 1.0000 - loss: 7.7465e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3747
Epoch 90/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 1.0000 - loss: 7.5732e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3743
Epoch 91/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 7.3981e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3743
Epoch 92/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 7.2275e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3740
Epoch 93/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 7.0650e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3741
Epoch 94/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 6.9124e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3734
Epoch 95/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 6.7509e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3732
Epoch 96/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 6.6011e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3729
Epoch 97/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 6.4574e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3726
Epoch 98/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 1.0000 - loss: 6.3220e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3723
Epoch 99/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 6.1840e-0
4 - val_accuracy: 0.8448 - val_loss: 0.3722
Epoch 100/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 6.0591e-0
4 - val_accuracy: 0.8276 - val_loss: 0.3721
```

```
2/2 - 0s - 23ms/step - accuracy: 0.8276 - loss: 0.3721
Test accuracy: 0.8275862336158752
```

## Visualize the Results

In [132…

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

epochs = range(1, len(history.history['loss']) + 1)

ax1.plot(epochs, history.history['loss'], label='Train Loss')
ax1.plot(epochs, history.history['val_loss'], label='Val Loss')
ax1.set_title('Loss')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.legend()

ax2.plot(epochs, history.history['accuracy'], label='Train Accuracy')
ax2.plot(epochs, history.history['val_accuracy'], label='Val Accuracy')
ax2.set_title('Accuracy')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy')
ax2.legend()

plt.tight_layout()
plt.show()
```



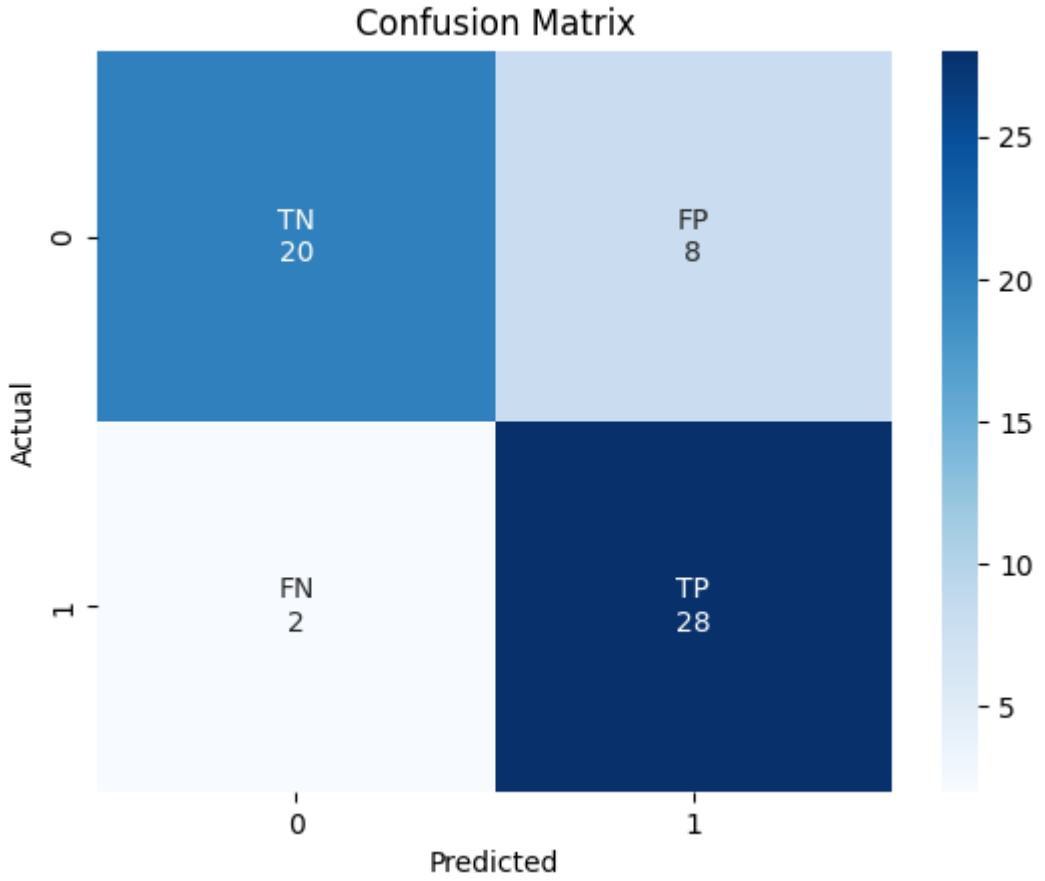

## Confusion Matrix

In [133…

```
y_pred = model.predict(test_data)
y_pred = (y_pred > 0.5)

cm = confusion_matrix(test_labels, y_pred)
print(cm)

tn, fp, fn, tp = cm.ravel()

labels = [
    [f"TN\n{tn}", f"FP\n{fp}"],
    [f"FN\n{fn}", f"TP\n{tp}"]
]

sns.heatmap(cm, annot=labels, fmt='', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
```

```
plt.show()
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step
[[20  8]
 [ 2 28]]
```




Eval Scores

In [134…

```
accuracy = accuracy_score(test_labels, y_pred)
precision = precision_score(test_labels, y_pred)
recall = recall_score(test_labels, y_pred)
f1 = f1_score(test_labels, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

```
Accuracy: 0.8276
Precision: 0.7778
Recall: 0.9333
F1 Score: 0.8485
```

## Conclusion

The model even though overfitting due to lack of data still learns as we see the training loss and the validation loss decreasing as epochs go on.

The oscillation behaviour in the validaiton accuracy occurs due to a smaller validation/test dataset as well. Each misclassification changes the accuracy significantly.

In the end we still acquire a good Accuracy given such a simple network and such less dataset

Note: Cruicial things in getting high enough accuracy lies in converting the images to binary pixels and balancing out the dataset. plus shuffling given how we are importing the dataset (folder by folder)

---

# Question 2

In [13]:
```
from scipy.linalg import ishermitian
```

## 1. Generating Data

### Define validation functions

In [14]:
```
def is_valid(matrix):

    eigenvalues = np.linalg.eigvals(matrix)
    trace = np.trace(matrix)

    cond1 = ishermitian(matrix)
    cond2 = np.all(eigenvalues >= 0)
    cond3 = np.isclose(trace, 1)

    return cond1 and cond2 and cond3

def make_valid_matrix():
    A = np.random.randn(2,2)
    rho = A @ A.conj().T
    rho = rho / np.trace(rho)

    return rho
```

### Generate Data

In [ ]:
```
valid_matrices = []
invalid_matrices = []

for _ in range(5000):
    matrix = make_valid_matrix()
    valid_matrices.append(matrix)

done = False
while not done:
    matrix = np.random.randn(2,2)

    if not is_valid(matrix):
        invalid_matrices.append(matrix)

    if len(invalid_matrices) >= 5000:
```

```
        done = True
```

## Split the data for training and testing

In [16]:
```
X = np.concatenate([valid_matrices, invalid_matrices])
y = np.concatenate([np.ones(len(valid_matrices)),
                    np.zeros(len(invalid_matrices))])

idx = np.random.permutation(len(X))
X = X[idx]
y = y[idx]

split = int(0.8 * len(X))

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]
```

# 2. Training the Neural Net

In [19]:
```
model_matrix = models.Sequential([
    layers.Input(shape=(2,2)),
    layers.Flatten(),
    layers.Dense(8, activation='relu'),
    layers.Dense(4, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model_matrix.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

history_matrix = model_matrix.fit(
    X_train,
    y_train,
    epochs=50,
    validation_data=(X_test, y_test)
)

test_loss, test_acc = model_matrix.evaluate(X_test, y_test)
print(f'Test accuracy: {test_acc}')

train_loss = history.history['loss']
train_accuracy = history.history['accuracy']
val_loss = history.history['val_loss']
```

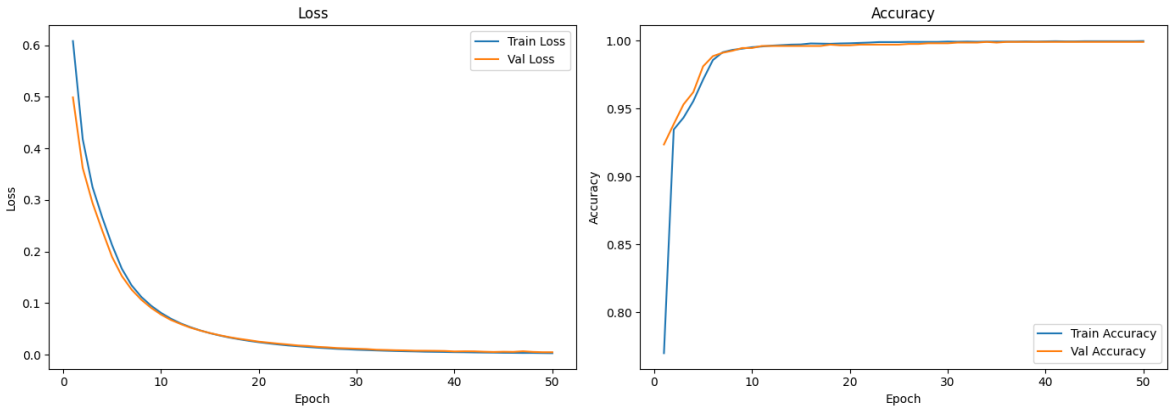
```
Epoch 1/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.7699 - loss: 0.6082
- val_accuracy: 0.9235 - val_loss: 0.4990
Epoch 2/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9345 - loss: 0.4179
- val_accuracy: 0.9385 - val_loss: 0.3621
Epoch 3/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9433 - loss: 0.3251
- val_accuracy: 0.9530 - val_loss: 0.2944
Epoch 4/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9554 - loss: 0.2661
- val_accuracy: 0.9620 - val_loss: 0.2406
Epoch 5/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9712 - loss: 0.2127
- val_accuracy: 0.9810 - val_loss: 0.1896
Epoch 6/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9856 - loss: 0.1667
- val_accuracy: 0.9885 - val_loss: 0.1526
Epoch 7/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9914 - loss: 0.1344
- val_accuracy: 0.9910 - val_loss: 0.1262
Epoch 8/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9931 - loss: 0.1120
- val_accuracy: 0.9925 - val_loss: 0.1065
Epoch 9/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9940 - loss: 0.0949
- val_accuracy: 0.9945 - val_loss: 0.0908
Epoch 10/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9951 - loss: 0.0812
- val_accuracy: 0.9945 - val_loss: 0.0780
Epoch 11/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9956 - loss: 0.0700
- val_accuracy: 0.9960 - val_loss: 0.0676
Epoch 12/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9962 - loss: 0.0609
- val_accuracy: 0.9960 - val_loss: 0.0597
Epoch 13/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9966 - loss: 0.0535
- val_accuracy: 0.9960 - val_loss: 0.0526
Epoch 14/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9970 - loss: 0.0473
- val_accuracy: 0.9960 - val_loss: 0.0471
Epoch 15/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9971 - loss: 0.0419
- val_accuracy: 0.9960 - val_loss: 0.0421
Epoch 16/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9979 - loss: 0.0373
- val_accuracy: 0.9960 - val_loss: 0.0378
Epoch 17/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9977 - loss: 0.0333
- val_accuracy: 0.9960 - val_loss: 0.0341
Epoch 18/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9976 - loss: 0.0300
- val_accuracy: 0.9970 - val_loss: 0.0309
Epoch 19/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9979 - loss: 0.0270
- val_accuracy: 0.9965 - val_loss: 0.0283
Epoch 20/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9980 - loss: 0.0243
- val_accuracy: 0.9965 - val_loss: 0.0254
```

```
Epoch 21/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9983 - loss: 0.0221
- val_accuracy: 0.9970 - val_loss: 0.0235
Epoch 22/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9985 - loss: 0.0200
- val_accuracy: 0.9970 - val_loss: 0.0216
Epoch 23/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9989 - loss: 0.0181
- val_accuracy: 0.9970 - val_loss: 0.0199
Epoch 24/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9989 - loss: 0.0166
- val_accuracy: 0.9970 - val_loss: 0.0180
Epoch 25/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9989 - loss: 0.0151
- val_accuracy: 0.9970 - val_loss: 0.0170
Epoch 26/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9990 - loss: 0.0138
- val_accuracy: 0.9975 - val_loss: 0.0154
Epoch 27/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9990 - loss: 0.0126
- val_accuracy: 0.9975 - val_loss: 0.0144
Epoch 28/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9990 - loss: 0.0114
- val_accuracy: 0.9980 - val_loss: 0.0131
Epoch 29/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9990 - loss: 0.0107
- val_accuracy: 0.9980 - val_loss: 0.0124
Epoch 30/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9992 - loss: 0.0098
- val_accuracy: 0.9980 - val_loss: 0.0117
Epoch 31/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9991 - loss: 0.0091
- val_accuracy: 0.9985 - val_loss: 0.0111
Epoch 32/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9992 - loss: 0.0084
- val_accuracy: 0.9985 - val_loss: 0.0098
Epoch 33/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9991 - loss: 0.0078
- val_accuracy: 0.9985 - val_loss: 0.0094
Epoch 34/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9992 - loss: 0.0073
- val_accuracy: 0.9990 - val_loss: 0.0088
Epoch 35/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9992 - loss: 0.0068
- val_accuracy: 0.9985 - val_loss: 0.0085
Epoch 36/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9992 - loss: 0.0064
- val_accuracy: 0.9990 - val_loss: 0.0079
Epoch 37/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9992 - loss: 0.0059
- val_accuracy: 0.9990 - val_loss: 0.0078
Epoch 38/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9994 - loss: 0.0057
- val_accuracy: 0.9990 - val_loss: 0.0077
Epoch 39/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9992 - loss: 0.0054
- val_accuracy: 0.9990 - val_loss: 0.0074
Epoch 40/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9994 - loss: 0.0051
- val_accuracy: 0.9990 - val_loss: 0.0064
```

```
Epoch 41/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9995 - loss: 0.0049
- val_accuracy: 0.9990 - val_loss: 0.0068
Epoch 42/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9994 - loss: 0.0046
- val_accuracy: 0.9990 - val_loss: 0.0067
Epoch 43/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9994 - loss: 0.0043
- val_accuracy: 0.9990 - val_loss: 0.0059
Epoch 44/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9995 - loss: 0.0042
- val_accuracy: 0.9990 - val_loss: 0.0055
Epoch 45/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9995 - loss: 0.0039
- val_accuracy: 0.9990 - val_loss: 0.0059
Epoch 46/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9995 - loss: 0.0037
- val_accuracy: 0.9990 - val_loss: 0.0056
Epoch 47/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9995 - loss: 0.0035
- val_accuracy: 0.9990 - val_loss: 0.0068
Epoch 48/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9995 - loss: 0.0036
- val_accuracy: 0.9990 - val_loss: 0.0056
Epoch 49/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9995 - loss: 0.0033
- val_accuracy: 0.9990 - val_loss: 0.0050
Epoch 50/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9996 - loss: 0.0031
- val_accuracy: 0.9990 - val_loss: 0.0050
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9990 - loss: 0.0050
Test accuracy: 0.9990000128746033
```

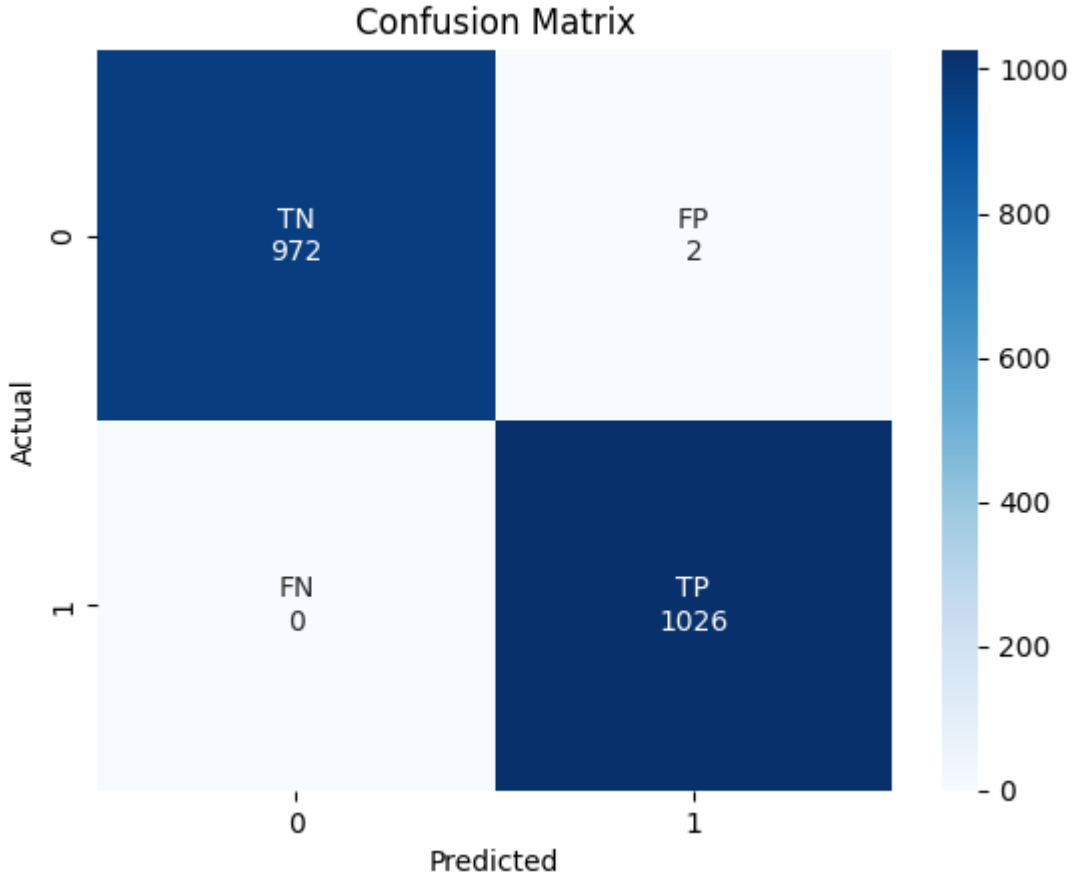
In [20]:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

epochs = range(1, len(history_matrix.history['loss']) + 1)

ax1.plot(epochs, history_matrix.history['loss'], label='Train Loss')
ax1.plot(epochs, history_matrix.history['val_loss'], label='Val Loss')
ax1.set_title('Loss')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.legend()

ax2.plot(epochs, history_matrix.history['accuracy'], label='Train Accurac
ax2.plot(epochs, history_matrix.history['val_accuracy'], label='Val Accur
ax2.set_title('Accuracy')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy')
ax2.legend()

plt.tight_layout()
plt.show()
```




In [21]:
```
y_pred_matrix = model_matrix.predict(X_test)
y_pred_matrix = (y_pred_matrix > 0.5)

cm = confusion_matrix(y_test, y_pred_matrix)
print(cm)

tn, fp, fn, tp = cm.ravel()

labels = [
    [f"TN\n{tn}", f"FP\n{fp}"],
    [f"FN\n{fn}", f"TP\n{tp}"]
]

sns.heatmap(cm, annot=labels, fmt='', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
[[ 972    2]
 [   0 1026]]
```




In [22]:
```
accuracy = accuracy_score(y_test, y_pred_matrix)
precision = precision_score(y_test, y_pred_matrix)
recall = recall_score(y_test, y_pred_matrix)
f1 = f1_score(y_test, y_pred_matrix)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

```
Accuracy: 0.9990
Precision: 0.9981
Recall: 1.0000
F1 Score: 0.9990
```

Even though the training is overfitting we still get 99% accuracy and good training and validation loss landscapes. This just shows how easy is the problem for FCN to solve